

KN-STORE

Plataforma E-commerce · Spring Boot + React + MongoDB

PLAN DE TRABAJO

Backend · Pagos, Facturacion, Auditoria y Calidad

Desarrollador	Laura
Rol	Backend · Pagos, facturacion, auditoria y calidad
Rama de trabajo	lauraG
Responsable tecnico	Joseph (integracion main)
Fecha de emision	25 de julio de 2026
Version	v1.0

1. Contexto y alcance

KN-Store ya construyo el nucleo transaccional, pero el dinero aun no fluye de verdad: el pago es simbolico (auto-aprobado), la factura no tiene consecutivo ni documento, y ninguna entidad de negocio registra quien la modifiko. Tu responsabilidad es convertir ese simulacro en una arquitectura seria: auditoria completa, precision monetaria, abstraccion de pasarela de pagos (con implementacion simulada configurable), reembolsos y facturacion interna real con PDF y QR.

Tu plan esta ordenado para que trabajes sin esperar: las tareas L-01 y L-02 no tienen dependencias y son aditivas (no rompen el trabajo de Santiago). Tus tareas centrales (L-03 en adelante) arrancan cuando Santiago entregue las transacciones reales (S-03) y el checkout consistente (S-07).

2. Resumen de tareas

ID	Tarea	Prioridad	RF / RNF	Depende de	Inicio
L-01	Auditoria e historial de estados	ALTA	RNF-025	—	Inmediato
L-02	Precision monetaria uniforme	MEDIA	RNF-026, RF-022	—	Inmediato
L-03	Abstraccion pasarela de pagos	ALTA	RF-054..056/058/059	S-03, S-07	Fase 2
L-04	Reembolsos	MEDIA	RF-057	L-03	Fase 2
L-05	Facturacion real PDF + QR	ALTA	RF-066..069	L-03	Fase 3
L-06	Tests y calidad SonarQube	MEDIA	Calidad	L-03, L-05	Fase 3

3. Orden de implementacion y detalle

FASE 1 · INICIO INMEDIATO — sin dependencias (L-01 » L-02)

L-01 · Auditoria de entidades de negocio e historial de estados

Prioridad: **ALTA** RF/RNF: RNF-025 Depende de: Ninguna

OBJETIVO

La infraestructura de auditoria existe (AbstractAuditingEntity + @EnableMongoAuditing) pero solo User la hereda. Ninguna entidad de negocio guarda quien la creo o modifiko, y no hay rastro de los cambios de estado de pedidos y pagos.

IMPLEMENTACION

- Extender AbstractAuditingEntity en: Pedido, Pago, Envio, Factura, Carrito, ItemCarrito, Producto, ProductoPrecio, ProductoInventario, Direccion y Cuenta.
- Crear el documento HistorialEstado { entidad, idEntidad, campo, valorAnterior, valorNuevo, fecha, actor } con su repositorio y servicio.
- Registrar las transiciones de Pedido.estado y Pago.estado, incluidas las que produce el checkout.
- Exponer GET /api/pedidos/{id}/historial y GET /api/pagos/{id}/historial (propietario, ADMIN o MANAGER).
- Coordinar con Santiago: el registrara las transiciones de sus endpoints admin (S-08) usando tu servicio. Acuerden la firma antes de que el implemente.

ARCHIVOS CLAVE

domain/*.java · domain/HistorialEstado.java (nuevo) · repository/HistorialEstadoRepository.java (nuevo) · service/HistorialEstadoService.java (nuevo)

CRITERIOS DE ACEPTACION

- Toda escritura en entidades de negocio persiste createdBy/createdDate/lastModifiedBy/lastModifiedDate.
- Cambiar el estado de un pedido deja un rastro consultable con fecha y actor.
- Tus cambios son aditivos (campos y clases nuevas): no rompen el trabajo en curso de Santiago.

Puede iniciar: De inmediato. Tus cambios no chocan con los de Santiago.

Desbloquea: [Insumo para S-08 \(Santiago registra transiciones con tu servicio\)](#).

L-02 · Precision monetaria uniforme

Prioridad: **MEDIA** **RF/RNF:** RNF-026 · RF-022 **Depende de:** Ninguna

OBJETIVO

Todo el dominio usa BigDecimal, pero sin escala uniforme al persistir, y la ganancia del precio no se calcula automaticamente.

IMPLEMENTACION

- Crear la utilidad MoneyUtils (service/util/) que aplique setScale(2, RoundingMode.HALF_UP) a todo valor monetario antes de persistir.
- Aplicarla en CheckoutService (subtotal, IVA, envio, total), ProductoPrecio, ItemCarrito, ItemPedido, Pago y Factura.
- Automatizar la ganancia en ProductoPrecioServiceImpl: ganancia = precioVenta - precioCompra, con su margen, al guardar (RF-022).
- No cambiar firmas de API ni formatos JSON: es un cambio interno de normalizacion.

ARCHIVOS CLAVE

service/util/MoneyUtils.java (nuevo) · service/CheckoutService.java · service/impl/ProductoPrecioServiceImpl.java

CRITERIOS DE ACEPTACION

- Ningun valor monetario persiste con mas de 2 decimales.
- Tests unitarios de MoneyUtils y del calculo de ganancia.
- CheckoutService queda listo para tu refactor de L-03.

Puede iniciar: De inmediato (en paralelo con L-01).

Desbloquea: [Base numerica confiable para L-03 y L-05](#).

FASE 2 · TRAS S-03 + S-07 EN MAIN (L-03 » L-04)

L-03 · Abstraccion de pasarela de pagos + flujo transaccional

Prioridad: **ALTA** **RF/RNF:** RF-054 · RF-055 · RF-056 · RF-058 · RF-059 **Depende de:** S-03 · S-07

OBJETIVO

Reemplazar el pago simbolico auto-aprobado por una arquitectura de pasarela real. Se implementa una pasarela simulada configurable; la pasarela concreta (Wompi, ePayco u otra) se enchufa despues sin tocar el dominio.

IMPLEMENTACION

- Definir la interfaz PaymentGateway: iniciarPago(pedido, metodoPago), consultarEstado(referencia), procesarCallback(payload) y reembolsar(pago, motivo).
- Implementar SimulatedPaymentGateway (propiedad knstore.payment.gateway.type=simulated): genera la referencia, deja el pago en PENDING y lo resuelve por callback; resultado configurable (approve/reject) para pruebas.
- POST /api/pagos/iniciar (RF-054): crea el Pago en PENDING asociado al pedido.
- POST /api/pagos/callback (RF-055): idempotente (la misma referencia no se reprocesa); valida la coherencia del monto pagado contra el total del pedido (RF-056) y resuelve APPROVED o REJECTED; registra en el historial (L-01).
- GET /api/pagos/{id} para el propietario (RF-059) y GET /api/pagos paginado para ADMIN (RF-058).

- Refactorizar CheckoutService: deja de crear el pago auto-aprobado; el pedido nace en PENDING y se confirma cuando el callback aprueba. Publica el nuevo contrato para Nicolas.

ARCHIVOS CLAVE

service/payment/PaymentGateway.java (nuevo) · service/payment/SimulatedPaymentGateway.java (nuevo) · web/rest/PagoResource.java · service/impl/PagoServiceImpl.java · service/CheckoutService.java · application.yml

CRITERIOS DE ACEPTACION

- Un pago nace en PENDING y solo cambia por callback.
- Un monto incoherente rechaza el pago y marca el pedido.
- Un callback duplicado no duplica efectos (idempotencia demostrada en test).
- El checkout sigue siendo transaccional sobre el replica set.

Puede iniciar: Cuando S-03 (transacciones reales) y S-07 (checkout estable) esten mergeados a main. A partir de aquí CheckoutService es tuyo.

Desbloquea: L-04, L-05 y N-07.

L-04 · Reembolsos

Prioridad: **MEDIA** **RF/RNF:** RF-057 **Depende de:** L-03

OBJETIVO

Permitir al ADMIN devolver el dinero de un pago aprobado, con rastro completo.

IMPLEMENTACION

- POST /api/pagos/{id}/reembolso (solo ADMIN): motivo obligatorio, estado REFUNDED, fecha del reembolso y registro en el historial.
- Regla de negocio: solo pagos en APPROVED; un segundo reembolso se rechaza con 400 y mensaje claro.
- Si el pedido estaba SHIPPED o DELIVERED, coordinar con el endpoint de devolucion de Santiago (S-08) para dejar trazabilidad cruzada.
- Implementar reembolsar en SimulatedPaymentGateway respetando la interfaz.

ARCHIVOS CLAVE

web/rest/PagoResource.java · service/impl/PagoServiceImpl.java · service/payment/SimulatedPaymentGateway.java

CRITERIOS DE ACEPTACION

- El reembolso deja rastro completo (quien, cuando, por que).
- El doble reembolso es rechazado con mensaje claro (RNF-013).

Puede iniciar: Cuando L-03 este mergeado.

Desbloquea: N-09 (boton de reembolso en el panel admin).

FASE 3 · SOBRE LA PASARELA (L-05 » L-06)

L-05 · Facturacion interna real: consecutivo, PDF con QR y envio

Prioridad: **ALTA** **RF/RNF:** RF-066 · RF-067 · RF-068 · RF-069 **Depende de:** L-03

OBJETIVO

La factura deja de ser simbolica: numero real consecutivo, documento PDF descargable con QR de validacion interna y envio automatico por correo.

IMPLEMENTACION

- Consecutivo FAC-000001 con coleccion contadora atomica (mismo patron del numero de pedido) (RF-067).
- Generar el PDF cuando el pago queda APPROVED (RF-066): datos de la tienda, cliente, items, totales, numero y fecha.
- Incrustar un QR (ZXing) con el payload de validacion interna: numero|fecha|total|hash SHA-256.
- Agregar al pom.xml las librerias OpenPDF (com.github.librepdf) y ZXing.
- GET /api/facturas/{id}/pdf que devuelve application/pdf (propietario, ADMIN o MANAGER) (RF-068).

- Enviar la factura por MailService al correo del cliente; marcar enviada=true y fechaEnvio.
- Regenerar el PDF bajo demanda (stateless, RNF-021): no guardar archivos en disco.

ARCHIVOS CLAVE

service/impl/FacturaServiceImpl.java · service/invoice/FacturaPdfService.java (nuevo) · web/rest/FacturaResource.java · service/MailService.java · pom.xml

CRITERIOS DE ACEPTACION

- Pago aprobado » factura con consecutivo generada y correo enviado (cubierto por MailServiceIT).
- El PDF descargado abre correctamente y el QR es legible.
- RF-069 (listado para ADMIN/MANAGER) sigue funcionando con los datos nuevos.

Puede iniciar: Cuando L-03 este mergeado (los datos de pago reales alimentan la factura).

Desbloquea: N-08 (Nicolas activa el boton de descarga).

L-06 · Tests de pagos/facturacion y calidad SonarQube

Prioridad: **MEDIA** **RF/RNF:** Calidad de software **Depende de:** L-03 · L-05

OBJETIVO

Dejar tus modulos con cobertura real y sin deuda critica.

IMPLEMENTACION

- PaymentGatewayTest: flujo simulado completo, callback idempotente, coherencia de monto y reembolso.
- FacturaServiceTest: consecutivo, generacion del PDF y envio por correo.
- PagoResourceIT y FacturaResourceIT para los endpoints nuevos (incluye control de acceso por rol y propiedad).
- Resolver los code smells y bugs de tus modulos reportados por SonarQube (sonar-project.properties ya esta configurado).

ARCHIVOS CLAVE

src/test/java/com/mycompany/knstore/service/ · src/test/java/com/mycompany/knstore/web/rest/

CRITERIOS DE ACEPTACION

- Cobertura de tus modulos mayor o igual a 80%.
- Sin hallazgos blocker/critical en SonarQube en tus archivos.

Puede iniciar: Cuando L-03 y L-05 esten mergeados.

Desbloquea: Cierre de calidad del backend.

4. Mapa de dependencias del equipo

Legenda: A » B significa que B necesita a A mergeado en main para iniciar.

- S-03 (replica set) + S-07 (checkout consistente) » L-03 (pasarela de pagos)
- L-03 » L-04 (reembolsos) y L-05 (facturacion real)
- S-04 (direcciones) » N-04 (formulario y predeterminada)
- S-06 (busqueda full-text) » N-06 (SearchPage)
- S-07 (reglas de totales) » N-05 (checkout consume backend)
- L-03 (estados de pago) » N-07 (flujo de pago en UI)
- L-05 (PDF de factura) » N-08 (boton de descarga)
- S-08 (endpoints admin) + L-04 (reembolsos) » N-09 (panel admin)
- L-01 (historial de estados) es insumo de S-08: Santiago registra transiciones con el servicio de Laura.

5. Reglas de coordinacion

- Ramas: trabajas exclusivamente en tu rama (lauraG). Sincroniza con origin/main al inicio de cada semana (merge, no rebase).
- Orden de integracion a main: santiago » lauraG » Nicolas. El backend publica primero; el frontend solo consume contratos ya integrados en main.
- Contrato primero: quien crea o modifica un endpoint publica en el PR un ejemplo de request/response y avisa al equipo antes de mergear.
- Archivos compartidos (avisa antes de tocarlos): SecurityConfiguration.java, application*.yml, knstore.jdl, pom.xml, routes.tsx, app.tsx, package.json.
- Propiedad transitoria de CheckoutService.java: Santiago hasta cerrar S-07; despues Laura para L-03. Nadie mas lo modifica mientras tanto.
- Commits en formato convencional (feat/fix/chore/test/docs), como el historial del repositorio.
- Todo PR exige la suite en verde: ./mvnw verify (backend) o npm test (frontend).
- Si dos tareas chocan por dependencia, mergea primero quien bloquea al otro.